

SKILL 24: VARIABLE REFERENCE DETECTION AND EXPANSION

Name: Teja Hasini Reddy

Roll No: 2520030048

Sec: 8

PART 1: ENVIRONMENT VARIABLE EXPANSION

Aim

To write a C program to detect variable references, expand their values from the environment, handle undefined variables, and display the expanded result.

Theory

Environment variables are variables maintained by the operating system and shell.

For example:

```
HOME=/home/user  
PATH=/usr/bin:/bin  
USER=student
```

A variable can be referenced using the \$ symbol.

For example:

\$HOME

The `getenv()` function retrieves the value of an environment variable.

Variable expansion replaces a variable reference with its actual value.

Example:

Input: Home is \$HOME

Output: Home is /home/user

If the variable does not exist, it is treated as an undefined variable.

Objectives

1. Read a string from the user.
2. Detect \$ variable references.
3. Extract the variable name.
4. Retrieve its value.
5. Replace the variable reference with its value.
6. Handle undefined variables.
7. Display the expanded string.
8. Test different environment variables.

Functions Used

Function	Purpose
<code>getenv()</code>	Retrieves environment variable value
<code>strstr()</code>	Searches for a string
<code>strcpy()</code>	Copies a string
<code>strcat()</code>	Joins strings
<code>printf()</code>	Displays output
<code>strlen()</code>	Finds string length

C Code

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

int main()
{
    char input[200], name[100], result[500] = "";
    int i = 0, j;
    char *value;

    printf("Enter text: ");
    fgets(input, sizeof(input), stdin);

    while (input[i] != '\0')
    {
        if (input[i] == '$')
```

```

{
    i++;
    j = 0;

    while (isalnum(input[i]) || input[i] == '_')
        name[j++] = input[i++];

    name[j] = '\0';

    value = getenv(name);

    if (value != NULL)
        strcat(result, value);
    else
        strcat(result, "[undefined]");
}
else
{
    char temp[2] = {input[i], '\0'};
    strcat(result, temp);
    i++;
}
}

printf("Expanded: %s", result);

return 0;
}

```

Explanation of Code

1. Read Input

```
fgets(input, sizeof(input), stdin);
```

Reads the text entered by the user.

2. Detect \$

```
if (input[i] == '$')
```

Checks whether the current character represents the beginning of a variable reference.

3. Extract Variable Name

```
while (isalnum(input[i]) || input[i] == '_')
```

Reads the characters forming the variable name.

For:

\$HOME

the variable name extracted is:

HOME

4. Get Variable Value

```
value = getenv(name);
```

Retrieves the value of the environment variable.

5. Handle Defined Variable

If the variable exists, its value is added to the output.

6. Handle Undefined Variable

If the variable does not exist:

[undefined]

is added to the output.

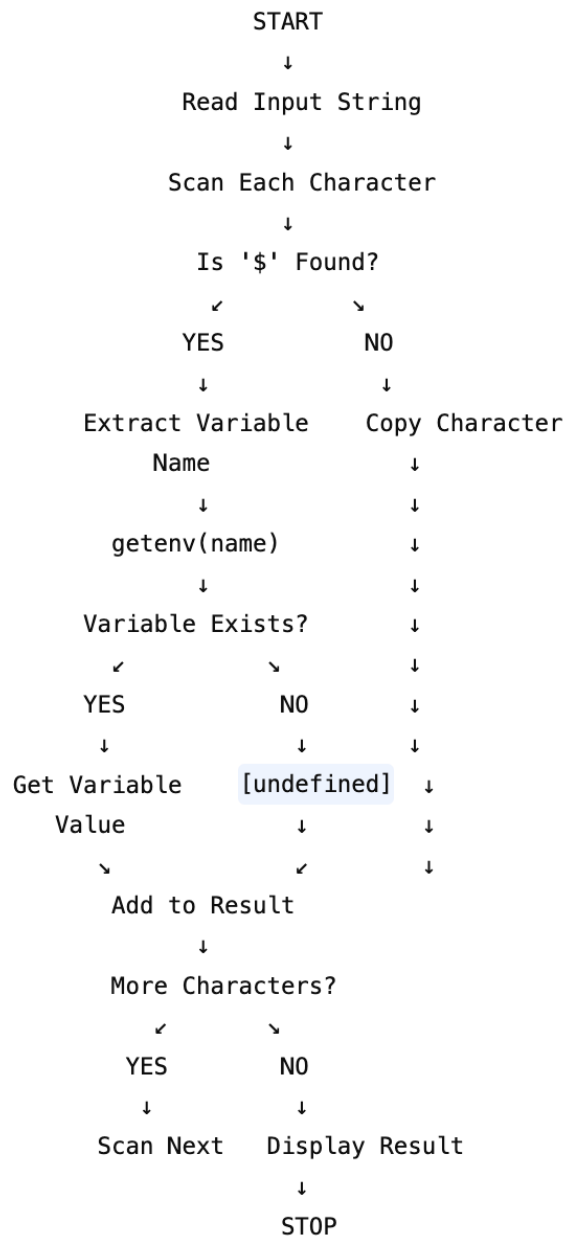
Compile

```
gcc variable.c -o variable
```

Run

```
./variable
```

Flow Chart



Sample Output – Defined Variable

Enter text: My home is \$HOME

Expanded: My home is /home/student

Sample Output – Multiple Variables

Enter text: User is \$USER and home is \$HOME

Expanded: User is student and home is /home/student

Sample Output – Undefined Variable

Enter text: Value is \$ABCXYZ

Expanded: Value is [undefined]

Observation

Operation	Observation
Input reading	User enters text
<code>\$</code> detection	Identifies variable reference
Variable extraction	Gets variable name
<code>getenv()</code>	Retrieves variable value
Defined variable	Value is expanded
Undefined variable	<code>[undefined]</code> is displayed
Result construction	Expanded text is generated

Test Cases

Input	Expected Output
Hello \$USER	Displays USER value
Home: \$HOME	Displays HOME value
Path: \$PATH	Displays PATH value
\$USER \$HOME	Expands both variables
\$ABCXYZ	Displays [undefined]
Hello World	Displays original text

Important Points

1. \$ indicates a variable reference.
2. `getenv()` is used to retrieve environment variables.
3. The variable name is extracted after \$.
4. Defined variables are replaced by their values.
5. Undefined variables are handled separately.
6. Multiple variables can be expanded in one input.
7. The program constructs the final expanded string.

OUTPUT

```
Last login: Thu Sep 17 11:32:46 on ttys000
7rainreddy7@Gunapatis-MacBook-Air ~ % >....

    value = getenv(name);

    if (value != NULL)
        strncat(result, value, sizeof(result) - strlen(result) - 1);
    else
        strncat(result, "[undefined]", sizeof(result) - strlen(result) - 1);
}
else
{
    char temp[2] = {input[i], '\0'};
    strncat(result, temp, sizeof(result) - strlen(result) - 1);
    i++;
}

printf("Expanded: %s", result);

return 0;
}
EOF
gcc variable.c -o variable && ./variable
Enter text: User is $USER and home is $HOME
Expanded: User is 7rainreddy7 and home is /Users/7rainreddy7
7rainreddy7@Gunapatis-MacBook-Air ~ % >....

    value = getenv(name);

    if (value != NULL)
        strncat(result, value, sizeof(result) - strlen(result) - 1);
    else
        strncat(result, "[undefined]", sizeof(result) - strlen(result) - 1);
}
else
{
    char temp[2] = {input[i], '\0'};
    strncat(result, temp, sizeof(result) - strlen(result) - 1);
    i++;
}

printf("Expanded: %s", result);

return 0;
}
EOF
gcc variable.c -o variable && ./variable
Enter text: Value is $ABCXYZ
Expanded: Value is [undefined]
7rainreddy7@Gunapatis-MacBook-Air ~ % █
```

Enter text: User is \$USER
Expanded: User is student

Result

The C program successfully detected environment variable references, extracted variable names, retrieved their values using `getenv()`, expanded defined variables, and handled undefined variables. Thus, variable reference detection and expansion were successfully demonstrated.